relation r). Given this input, the forward pass produces $T \times L$ layer outputs (one representation per T tokens and L layers). The algorithm aims to estimate the amount of information about the fact (s, r, o_{true}) that is contained in each of these representations. We denote the representation at token t and layer ℓ as $v_{(t,\ell)}$.

The amount of factual information in $v_{(t,\ell)}$ is estimated by copying this representation into a different forward pass obtained from using a noised subject in the input:

$$\text{Tracing Effect} = p_\theta(o_{true}|s_{noise}, r, v_{(t,\ell)}) - p_\theta(o_{true}|s_{noise}, r)$$

where s_{noise} indicates that we add Gaussian noise with $\sigma = 0.094$ to the token embeddings of s following Meng et al. [21], and $v_{(t,\ell)}$ is the representation at token t and layer ℓ in the forward pass on the original prompt $P = (s, r)$. The probability $p_\theta(o_{true}|s_{noise}, r, v_{(t,\ell)})$ is computed by (1) running the model forward pass on the noised prompt $P^* = (s_{noise}, r)$ until layer ℓ, (2) *overwriting* the existing representation at token t and layer ℓ with the representation $v_{(t,\ell)}$, then (3) computing the remaining $L - \ell$ layers as normal using this adjusted set of T representations as input (adjusted at token index t). Thus, Causal Tracing estimates the information content of a representation in terms of its effect on the probability of the true target. The results from Causal Tracing show where the representations containing information about the true target are in the model forward pass.

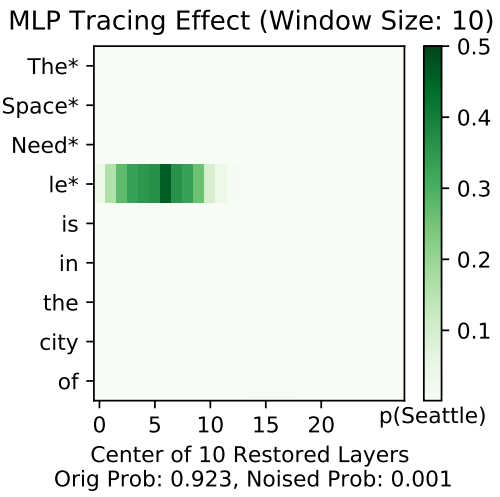


Figure 2: Visualizing Causal Tracing results over MLP layers with window size 10. Tokens with an asterisk are the noised subject tokens. Here, $p_\theta(o_{true}|s, r)$=.923 and $p_\theta(o_{true}|s_{noise}, r)$=.001.

In practice, a *set* of representations from multiple adjacent layers is copied from the clean forward pass rather than a single layer's representation (for instance, ten layers in Fig. 2). The size of this set is referred to as the *tracing window size*. A window size of, e.g., three implies that the tracing effect at layer ℓ estimates the amount of information contained in the three representations $v_{(t,\ell-1)}$, $v_{(t,\ell)}$, and $v_{(t,\ell+1)}$. See Appendix Figs. 10 and 11 for analysis of the parameter's effect. In this paper, we use a tracing window size of 5 by default, and we apply Causal Tracing exclusively to MLP layers, given evidence of their role in factual association [12, 21].

3.3 Model Editing with ROME

We describe the ROME editing method here since we use it in our analysis in Sec. 4, and later in Sec. 5 we outline additional editing methods we consider. For mathematical detail, see Meng et al. [21].

CounterFact Example

Input Prompt:	Autonomous University of Madrid, which is located in
Requested Edit:	Spain ⟶ Sweden
Paraphrase:	and Sallie Beavers Riley. Autonomous University of Madrid is located in
Neighbor:	Ripollès, located in

Figure 3: An example CounterFact datapoint.

The input to ROME includes a prompt $P = (s, r)$ and a new desired output, which is always a false target o_{false} in the CounterFact dataset. To change the model prediction to o_{false}, ROME applies a rank one edit to the down-projection matrix in a prespecified MLP layer in the model. The default layer in GPT-J is layer 6, following from averaged Causal Tracing results. ROME also makes use of covariance statistics of different subject representations obtained from a larger corpus as it edits individual facts. Overall, the method is designed to optimize the quantity $p_\theta(o_{false}|s, r)$ while aiming to satisfy some other constraints reflecting what a desirable model edit is (described in Sec. 3.4 next).

3.4 Editing Metrics

Editing methods are typically evaluated according to their ability to (1) change the model prediction on the input P provided at runtime, (2) generalize appropriately to paraphrases of the prompt P, and (3) avoid over-generalizing to unrelated data [42, 9, 24, 15, 25]. We adopt metrics for each desideratum that we compute with available CounterFact data. Instead of the exact "magnitude" metrics from Meng et al. [21], we use normalized versions of each metric that we design to scale from 0 to 1 depending on whether the edit was maximally (un)successful, for purposes of making scores more comparable across data points. We denote the new edited weights of the LM as θ^* and its pre-edit weights as θ. See Fig. 3 for an example of the kinds of data these metrics are computed on.

1. *Rewrite Score*. The rewrite score measures how much an edit improves the target probability $p(o_{false}|s, r)$ as a fraction of the maximum possible improvement:

$$\frac{p_{\theta^*}(o_{false}|s, r) - p_\theta(o_{false}|s, r)}{1 - p_\theta(o_{false}|s, r)}$$

2. *Paraphrase Score*. The paraphrase score measures the target probability using syntactical paraphrases as inputs, always preserving the exact subject wording:

$$\frac{p_{\theta^*}(o_{false}|s, r^*) - p_\theta(o_{false}|s, r^*)}{1 - p_\theta(o_{false}|s, r^*)}$$

   which is averaged over multiple available paraphrases per input P. The score measures whether edits properly generalize across semantically equivalent prompts.

3. *Neighborhood Score*. The neighborhood score measures whether edits change predictions for prompts with a similar subject s^*, the same relation r, and the same (true) objects. We scale the difference in probabilities so that 1 means the probability did not change (good), and 0 means it changed to the maximum extent possible (bad):

$$1 - \frac{|p_{\theta^*}(o_{false}|s^*, r) - p_\theta(o_{false}|s^*, r)|}{.5 + |p_\theta(o_{false}|s^*, r) - .5|}$$

   The score measures whether edits avoid *over*-generalizing from the prompt P to different subjects.

4 Does Edit Success Follow From Localization?

Ostensibly, localization results should inform editing methods because it should help to know where information is stored in a model if you are going to manipulate the model's expression of that information. More specifically, if you wanted to inject a false belief (s, r, o_{false}) into a model (as defined in the ROME editing problem), it seems helpful to know which weights store the true fact (s, r, o_{true}), so that you could replace some stored representation of o_{true} with that of o_{false}. This underlying assumption about editing models appears in much past work on localization, where editing is used to verify localization analysis (see Sec. 2). In this section, we investigate the validity of this assumption as it applies to autoregressive Transformers.

4.1 Experiment Design

The goal of our experiments is to determine, for a given datapoint, whether edit success *at a specific layer* aligns with the results from Causal Tracing at that layer (see Causal Tracing description in Sec. 3.2). We operationalize this outcome and explanatory variable as follows:

1. *Edit Success*. We primarily consider Rewrite Score as our measure of edit success, given that this is the main optimization objective of ROME. Note ROME achieves an average rewrite score of 99% at layer 6 of GPT-J and above 96% at layers besides the last layer of the model.
2. *Tracing Effect at layer ℓ*. Since the output of Causal Tracing is a $T \times L$ grid of estimates, we obtain a single tracing effect per layer by taking the max across the T token effects at each layer (i.e., we collapse the grid in Fig. 2 down to a single curve across layers). Like our other metrics,

we use a *fractional* tracing effect where 0 means the intervention had no effect and 1 means it fully restored the original probability $p_\theta(o_{true}|s, r)$:

$$\frac{p_\theta(o_{true}|s_{noise}, r, v_{(t,\ell)}) - p_\theta(o_{true}|s_{noise}, r)}{p_\theta(o_{true}|s, r) - p_\theta(o_{true}|s_{noise}, r)}$$

Lastly, note we use a tracing window size of 5 (smaller than the value of 10 used in Fig. 2).

4.2 Model and Data

We conduct our analysis with GPT-J [35] using the CounterFact dataset, similar to Meng et al. [21]. GPT-J is a 6 billion parameter autoregressive language model. We record editing performance at layers in {1, 5, 9, 13, 17, 21, 25, 28} as well as layer 6 (the default for ROME). Note ROME achieves an average rewrite score of 99% at layer 6 and above 96% at layers besides layer 28.

The CounterFact dataset includes datapoints consisting of a prompt, paraphrases, and neighboring points. For each point, a new (false) target is supplied for editing purposes. We show an example datapoint in Fig. 3. Note paraphrases intentionally include unrelated text preceding a syntactical paraphrase of the input, with the idea that this text should not affect the output. We select data for experiments from 10% of CounterFact, additionally filtering to a subset of facts that are correctly completed by GPT-J, in order to ensure that there is knowledge to localize in the model for each point (details in Appendix A). Our final sample size is $n = 652$.

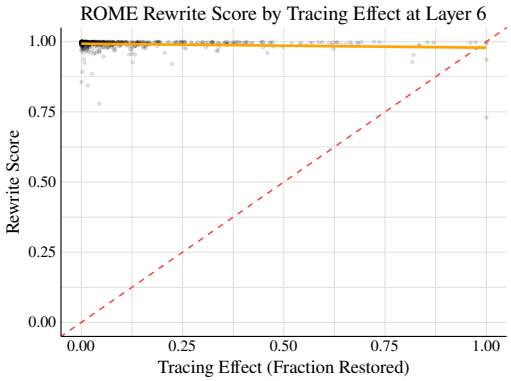


Figure 4: The correlation between ROME edit success and the tracing effect at layer 6 in GPT-J is not positive but in fact slightly negative ($\rho = -0.13$; p <1e−3). The dashed red line shows a hypothetical perfect relationship.

4.3 Experiment Results

We present results in two ways. First, in Fig. 4, we show Rewrite Score as a function of the (fractional) tracing effect. The red dotted line shows a hypothetical perfect relationship between tracing and edit success. Surprisingly, there is not a positive relationship but a *negative* relationship between the rewrite score and the tracing effect (linear correlation of $\rho = -0.13$; p <1e−3). This seems to fully invalidate the assumption that editing should be most effective when it occurs at a layer where information is stored about the edited fact. We wish to emphasize, however, that in most layers we simply see a near-zero rather than negative correlation, as shown in Appendix Fig. 15.

Our second mode of analysis is though linear regression models predicting rewrite score based on (1) the tracing effect, (2) the choice of edit layer treated as a categorical variable, or (3) both terms interacted, again treating edit layer as a categorical variable. The purpose of the models is to show how much of the variance in rewrite score is explained by one variable versus the other. We show the resulting R^2 values in Table 1. We see that the choice of layer explains almost all of the variance in rewrite score (94.7%), while adding the tracing effect to the model raises the R^2 only to 94.8%. This means that **the tracing effect is able to explain only 0.1% of the variance in edit success** when accounting for the choice of edit layer. These results suggest that the tracing effect is essentially unrelated to the success of model editing.

Table 1: R^2 values for predicting ROME edit success. Tracing effects explain essentially none of the variance in rewrite score, while the choice of edit layer is very important.

	R^2 Values		
Method	Layer	Tracing Effect	Both
ROME	0.947	0.016	0.948

This is a surprising conclusion, and it naturally raises the question of why applying ROME at layer 6 works well in the first place (see average rewrite, paraphrase, and neighborhood scores across layers in Appendix Fig. 7). We suggest a possible answer to this question in Sec. 6.